Milestone 4: Executive Dashboard & Production Deployment

1. Milestone Objective

The final phase of the pipeline focuses on data consumption. The objective was to build a secure, interactive web application that reads the data processed by Airflow (salesorder_cleaned.csv) and transforms it into actionable business intelligence for executive stakeholders.

2. Technology Stack

- **Frontend Framework:** Streamlit
- **Data Manipulation:** Python (Pandas)
- **Data Visualization:** Plotly Express
- **Deployment & Hosting:** GitHub, Streamlit Community Cloud
- **Security:** Streamlit Secrets Management

3. Core Features & Implementation

We developed a highly resilient dashboard designed to handle real-world data discrepancies without crashing. Key implementations include:

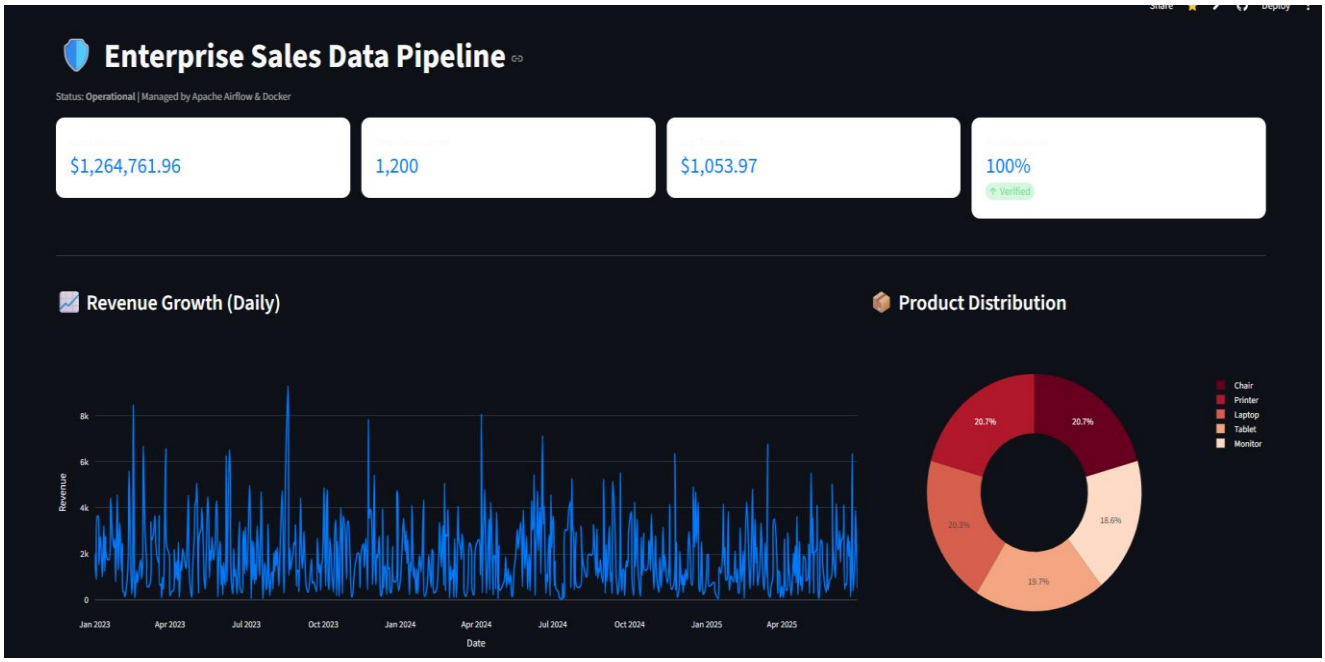
- **Dynamic Column Mapping (Fuzzy Logic):** Instead of hardcoding exact column names, the data engine uses fuzzy logic to identify columns. For example, it successfully parses variations like "Total amount ", "totalvalue", or "totalprice", preventing fatal KeyError crashes in production.
- **Executive KPI Cards:** Designed custom CSS to present key metrics (Gross Revenue, Total Transactions, Average Ticket Size) in a professional enterprise theme (Deep Blue #1E3A8A).
- **Time-Series Aggregation:** Implemented Pandas resampling (dt.to_period('M')) to aggregate daily transactional noise into a smooth, readable **Monthly Revenue Growth** trendline using Plotly Area charts.
- **System Audit Log:** Included an expandable data inspector tab that allows administrators to audit the underlying CSV data directly from the UI.

4. Security & Deployment Architecture

Deploying the application from a local Docker environment to the public cloud required strict environment and security configurations:

- **Dependency Management:** Created a strict requirements.txt file (streamlit, pandas, plotly) to ensure the Streamlit Cloud container built successfully without ModuleNotFound errors.
- **Environment Agnostic Paths:** Refactored Python file paths using os.path.exists() and relative routing so the script operates flawlessly whether executed in GitHub Codespaces, a local machine, or the Cloud server.
- **Secrets Isolation:** Ensured all sensitive credentials (like Gmail App Passwords used in earlier Airflow alerts) were secured via .gitignore and migrated to **Streamlit App Secrets** for the live environment.

5. Challenges & Solutions

During the deployment phase (Sprint 4), the team overcame the following technical hurdles:

1. **Challenge:** The cloud server crashed upon deployment because the Pandas .dropna() function could not locate the hardcoded "Date" column due to hidden trailing spaces in the CSV.
    - **Resolution:** Engineered a str.strip() cleaning pass and a dynamic search function to locate and standardize column headers before rendering the UI.
2. **Challenge:** The default daily line chart was too volatile for long-term business analysis.
    - **Resolution:** Applied Pandas grouping logic to group the DataFrame by month, delivering a much cleaner "Executive Level" area chart.